GB Notion Frontend — Basket Reporting

An offline version of the Basket Reporting form for use in the basket during a gas balloon flight. Same four message types as the Tally form, same submit action, but it keeps working without signal, stamps every entry with time and GPS position, carries the ballast figure forward across all devices, and lets several people report in parallel.

Every entry is written to a JSON and a CSV file in a GitHub repository. Notion polls those files. A manual entry is also sent as one JSON object to the configured Reporting Webhook. The app reads the GitHub files back every five seconds, so each device always shows the complete flight.

No build step and no dependencies. The app talks to the GitHub API, the configured Zapier webhook, OpenStreetMap when place names are enabled, and WhatsApp links when selected.

---

1. The screen

The whole app fits one phone screen without scrolling, and its outline is always visible. Only the log and the setup scroll.

Day mode is dark anthracite on a grey ground; unselected buttons carry a visible outline and a white face. Night mode is red on black and preserves dark adaptation. The sun/moon button in the header switches in one tap and the choice is remembered; on first launch the app follows the iPad's own appearance setting.

The header shows UTC to the second and, at its right, the ballast still on board in the same size, then the flight and callsign with the live navigation line beneath them — `293° · 12.3 kn · ± 8 m`, course over ground, ground speed and the accuracy of the current fix. Before a fix it reads `GPS searching`; with a fix but no movement the course and speed fall away and only the accuracy remains. Then two buttons: menu and minimise. Below it, flush against the header, sit the four message types; the kilograms on board appear under them only on the Ballast tab, where they are relevant. There is no logo and no title inside the app — the home screen icon already says what this is, and the room is better spent on the form.

Everything that is not reporting lives behind the menu: Report, Log, Settings, the day/night switch, Reload from GitHub, Send now, Print the log, Export the table, and *About — more info*, which opens this manual as a page in its own window; the licence is linked from its last section. *Report* is the filled button at the top, because reporting is what the app is for and the menu should not make you hunt for the way back. The switch is named for the mode it takes you to, so it reads *Night mode* by day and *Day mode* at night.

Keeping the header to two buttons leaves the whole width for the flight and the navigation line, and the three screens are always one tap apart through the menu.

Night mode comes in four colours — red, amber, green or dimmed white — chosen in the setup. Red preserves dark adaptation best; the others are there for personal preference and for screens where red reads badly.

The app icon and the favicon are the tally itself — four navy strokes struck through in red, drawn slightly out of true so it reads as counted by hand rather than printed. The favicons have a transparent ground so they sit in a browser tab of any colour; the home screen icons stay opaque, since iOS renders transparency in an app icon as black. The set covers all three platforms: `apple-touch-icon` in 120, 152, 167 and 180 pixels for iPhone and iPad, manifest icons in 192 to 512 for Android and Chrome, two maskable versions with the artwork inside the safe circle so Android's adaptive shapes do not clip it, a `mask-icon.svg` for pinned tabs in desktop Safari, and a multi-resolution `favicon.ico`. The icons are opaque — iOS renders transparency in a home screen icon as black — and the manifest background is white to match, so the Android splash shows no seam. The 16 and 32 pixel favicons carry a simplified version with three uprights instead of four; at that size the narrower gaps of the full mark close up into a smudge.

2. Set up

The GitHub side — the data repository, the token and the Notion polling — is a separate document: **Setup guide**. What follows is the short version.

Hosting

1. Put these files in the repository root.
2. Settings → Pages → Source: `Deploy from a branch`, branch `main`, folder `/ (root)`.
3. The app is served at `https://<owner>.github.io/<repo>/` after about a minute.

Token

Settings → Developer settings → Personal access tokens → Fine-grained tokens:

- Repository access: the data repository only
- Permissions → Repository permissions → **Contents: Read and write**
- Short expiry, for example one week after the flight

In the app

The *VFR squawk* field is the conspicuity code the VFR button writes; `7000` unless your airspace uses something else.

The *Ballast on board at start* field is only needed before the first inventory: it seeds the running figure so that early drops count, and it gives the first inventory something to compare against in `tally_diff_kg`. Leave it empty and the ballast figure simply starts with the first Take Inventory.

Device mode, language, reporter name, night colour and confirmation tone are personal settings and need no password. The shared flight setup below them is editable only on the Master: press *Unlock settings* and enter **1234**. Fill in the flight, the two pilots, the weight of one ballast bag, the full ready-ballast weight, and the quick drop amounts. The methanol level at the start of the flight is a setting of its own — a tank is not always full when the balloon leaves — and it is what the methanol button shows until the first Resources entry says otherwise.

The oxygen cylinders are a table of up to four, each with its volume and its pressure when full. Every input has its name above it and its unit inside it.

The **Reporting-Webhook** is the HTTPS endpoint that receives each manual entry for forwarding to Notion. Its default is `https://hooks.zapier.com/hooks/catch/21180853/4t6kt0g/`. It is part of the shared flight setup: only the Master can edit it, and Followers receive the Master's saved value automatically. Leaving the field empty disables webhook delivery.

Opening the **WhatsApp message layout** or the **GitHub connection** asks first — *Do you really want to change these settings?* Those two are where a slip costs a flight rather than a keystroke.

The GitHub connection and the WhatsApp recipients sit in collapsed sections at the bottom — open them once per device, then close them again. Saving asks twice before it applies, so a flight in progress cannot be overwritten by a stray tap. Leaving the setup screen locks it again.

3. Install on the iPad

Safari → Share → **Add to Home Screen**. It is saved as *Basket Reporting* and launches full screen without Safari's bars, and location and wake lock work as expected. Location permission is requested on the first entry; "While Using the App" is enough. Without it, entries are still recorded, with `gps_fix` set to `false`.

4. Floating over other apps

The minimise button collapses the app to a blue pill reading `HB-QWV Reporting` with the ballast still on board beneath it. Where that pill can float depends entirely on the platform, and it is worth being exact about it.

No web app can draw over other apps on iPadOS, iPhone or Android. That capability is reserved for native apps — on Android it needs the `SYSTEM_ALERT_WINDOW` permission, on iOS it does not exist at all. A browser page, installed or not, cannot paint outside its own window. Anything claiming otherwise on a web page is drawing inside that page.

So the pill floats inside the app window, and the app window is what the platform puts on top:

Platform	How to keep it above the other app
iPad	Install to the Home Screen, then drag it from the Dock onto the running app — **Slide Over** gives a narrow window floating over a map or Foreflight, swiped off the edge and back with one gesture. **Split View** and **Stage Manager** are the fixed and the free-floating variants.
Android, Samsung	Install to the Home Screen, then open it in **Pop-up view** from the Edge panel or the recents menu. An installed web app counts as an app there and gets a real floating window.
Android, others	**Split screen** from the recents menu. Free-floating windows exist only on Samsung and on devices in desktop mode.

Platform	How to keep it above the other app
Chrome on a laptop	The minimise button opens a genuine always-on-top window through document picture-in-picture — the only browser feature that does this. It survives switching to any other application. Useful for the chase car.

The app detects the picture-in-picture case on its own: press minimise and you get the always-on-top window where the browser supports it, and the in-app pill everywhere else. Clicking either brings the app back.

5. Who is reporting

The setup asks whether this is a **personal device** or a **shared device**.

On a personal device — the default, and what the two pilots' own phones should be set to — every entry is filed under the name entered there, whoever happens to be pilot in command. The reporter button is hidden; there is nothing to get wrong.

On a shared device the reporter defaults to whoever is pilot in command, and a button beside *Post to CC Notion* — about a third of the width — switches to the other pilot for the exceptional case. It reads *reporting as* with the name below, and appends *(PIC)* when the two are the same person, so a glance tells you both facts at once. It turns dark while the alternate is active, so the exception is visible, and every entry carries that name until it is switched back. Changing the PIC on the Resources tab moves the default with it.

Each row also records `device_mode`, so it is possible to tell afterwards which reporting regime a row came from.

5a. Position

The position on every row is the fix of the device that made the entry, taken at the moment of posting. Rows merged in from another iPad keep that iPad's position — nothing is ever re-stamped. Three columns carry it in short notation: `pos_lat` as `484532N`, `pos_lon` as `0072345E`, and `alt_ft` as WGS84 altitude in feet. The decimal degrees and metres stay alongside for map links and for anything that needs the raw value.

Track and ground speed come with every entry. iOS and Android supply both with the fix once the device is moving; when they do not, the app derives them from the previous fix, requiring at least three seconds and five metres between the two so that a stationary basket does not produce a random course. Both fields stay empty rather than guessing when neither source is available. The header carries the live values under the flight number.

6. Ballast

The kilogram field starts empty every time the Ballast tab is opened. A running total that carried over from ten minutes ago would be added to by mistake far more often than it would save a keystroke.

Sand and water are carried forward separately. An inventory sets both, a drop reduces the one it was made of, and every entry therefore knows what is left of each — recorded as `sand_left_kg` and `water_left_kg` beside the total. An inventory reads `sand 390 kg („Schütte" 100%) · water 20 kg · 24 bags` — the Schütte named separately as a percentage and counted inside the sand figure, since that is where it physically is. A drop reads `dropped 40 kg sand · remaining 370.0 kg total ballast (350.0 kg sand, 20.0 kg water)`, and a water drop is stated in litres, `dropped 20 l water`, because that is how it is measured in the basket.

Drop subtracts what was thrown and records whether it was sand or water; sand is the default. The quick amounts **add up**: tapping 10 three times gives 30, which is how ballast actually leaves a basket — sack by sack, counted as you go. A *Reset* button appears beside the total the moment there is something to undo. The `kg` at the right edge of the button row names the unit once, so the buttons themselves stay bare numbers. The quick buttons only fill the field; posting is always the one dark button, so a knock against the iPad cannot log a drop.

Take Inventory records what is actually on board and resets the running figure. Three inputs:

- *Bags* and *Water* on one line — bags counted in total including safety ballast and multiplied by the weight per bag from the setup, water in litres at one kilogram per litre.
- *Ready ballast* („Schütte") — the steps come from the setup, `0,10,25,50,75,100` by default, as a percentage of the full ready-ballast weight, normally 30 kg. The last value used stays selected.

The running total under the fields shows the result before it is posted, and the entry lands in the table as three columns:

Column	Contents
`sand_kg`	bags × weight per bag, **plus** the ready ballast
`water_kg`	litres
`total_ballast_kg`	sand + water, and the new figure on board

Ready ballast is always sand, so sand + water = total. `inv_bags` and `inv_ready_pct` are carried alongside so an inventory can be reopened and corrected later.

`tally_diff_kg` **records where the figures parted company.** Every inventory stores the counted total minus what the running calculation expected at that moment. A reading of −60 means sixty kilograms left the basket without being logged; a positive figure means a drop was logged that did not happen, or a bag was miscounted. The first inventory has no predecessor to compare against and leaves the column empty unless a start ballast was entered in the setup. The figure is recalculated over the whole flight after every edit, deletion or merge, so it always reflects the current state of the table. The app also shows the expected value and the difference live, before an inventory is posted.

The dropped figure is measured from the first inventory, not from the sum of the drops: `first inventory total − currently on board`. Throwing sand is not exact; counting sacks is. Every later inventory therefore feeds straight into the dropped figure, including whatever went overboard without being logged.

7. ATC, Resources, Other

ATC puts the station on its own line, then FREQ, SQ and a VFR button side by side, then the message.

A frequency must lie between **118.000 and 136.975 MHz** and be a real channel name. Both spacings are accepted: the 25 kHz channels ending in `00`, `25`, `50`, `75`, and the 8.33 kHz channels inserted between them, ending in `05 10 15`, `30 35 40`, `55 60 65`, `80 85 90`. The four endings `20`, `45`, `70` and `95` exist in neither scheme — `118.020` is neither a frequency nor a channel number — so they are refused as what they almost always are, a mistyped digit.

The **frequency** places its own point: after the third digit one appears as you type, so `12345` reads `123.45` on screen while you are still typing and settles to `123.450` when you leave the field. `1187` becomes `118.700`, `118` becomes `118.000`. A point or comma you type yourself is ignored — the field only ever counts digits — so both habits work.

The **squawk** is four octal digits — 8 and 9 simply do not appear as you type, and a code of one to three digits is refused on posting. The emergency codes are listed in the setup, `7500`, `7600` and `7700` by default; entering one brings up a confirmation naming what it means before the entry goes out, because they are three keystrokes away from ordinary codes.

The **VFR** button fills the squawk with the conspicuity code from the setup, `7000` by default, and lights up while that code is set. Tapping it again clears the field; typing anything else turns the light off. Station, FREQ and SQ open holding the last call in grey. Tapping into any of them clears that field at once — those three are replaced rather than edited, and deleting four digits by hand in turbulence is a waste of a hand. Typing turns the text dark.

Beside *Transmitted* sits **Copy from last call**, greyed and dashed, and it stays there. One tap puts all three values back in dark type and drops the cursor into the message, without any message of its own — the fields having filled is the confirmation. The station comes back with a trailing space and is then left alone: tapping into it keeps what was copied, so `ZURICH INFO` can be turned into `ZURICH INFO ARRIVAL` without retyping the first two words. FREQ and SQ still clear on a tap, because those two are replaced rather than extended.

Beneath the station field are the station presets — *Info, Radar, Tower, Control, Approach* by default, and whatever you put in the setup instead. A tap inserts the word **where the cursor stands** rather than replacing the field, with exactly one space between it and what is already there, so `ZURICH` + *Info* + *Radar* becomes `ZURICH Info Radar` and never `ZURICH Info`.

A message preset is inserted with a trailing space, so the next word can simply be typed on.

The message presets sit under the message field and **differ by direction**: one list for calls received, another for calls transmitted, both in the setup under *ATC*. What you note down when a station calls you is rarely what you note down when you call it. Both start with the same list until you change them.

The standard phrases sit above the message field. Tapping one adds it, tapping it again takes it out, so QNH and Ops Normal are a toggle rather than something that accumulates.

The remark field on Ballast and Resources is the same size as the ATC message field, since a remark is often the more important part of the entry.

Resources puts current PIC and who is resting on the first line as panel switches, then battery, fuel cell and solar cell on the second, then methanol level, *Crew on O2* and the O2 level on the third. Only pilots the master actually named appear: leave the second name empty and the switches shrink to a single-handed flight.

Changing the PIC puts the other pilot on rest automatically, because that is what a handover normally means. *Nobody* is still one tap away for the stretches when both are awake.

Battery and *Methanol level* both open the same list of percentages in a window, rather than a dropdown, so the two read alike and both are one tap away in gloves.

O_2 Reserve is calculated rather than typed: volume × pressure, summed over the cylinders, which gives the litres of gas at one bar. The button reads `600 l (21%) ②` — the reserve, its share of a full load, and the number of the bottle in use in a circle. It never shows more than 100 per cent, and it never shows pressures: those belong in the input window, which is where you report them. Tapping it opens one row per cylinder with its size, a field for the pressure, and a radio button marking which cylinder is **in use**.

The button only responds while **Crew on O_2** is on. Reporting a pressure while nobody is breathing from the bottle records a number that means nothing, and the reserve figure is only of interest once it is being consumed — battery as a list in ten-percent steps with 75, 85 and 95 added, where the reading actually matters. The PIC and resting state stays active and is attached to every following entry of any type; battery, fuel cell and solar belong to the entry they were filed with.

Opening the Ballast tab always lands on **Drop** with **Sand** selected. Those are the overwhelming majority of entries, and a form that opens on Inventory because that is what happened an hour ago costs a tap every single time.

Other is a free note, up to four pictures, and a voice note. They are scaled down to 1024 pixels and about 60 per cent JPEG quality in the browser before anything is stored, because a queued entry has to survive in the device's local storage until there is a link again. Each picture is written to `data/media/<entry-id>-<n>.jpg` in the repository and the row records how many were attached and where they landed.

Add voice msg records through the microphone — press once to start, once again to stop, two minutes at most. The recording appears with a player so it can be listened to before posting, and is written to `data/media/<entry-id>-voice.webm`, or `.m4a` on iOS, where Safari records in that format instead.

Attachments are not queued. Unlike the entries themselves, pictures and voice notes live only in memory until they are uploaded; posting without a connection files the note and says so, but the attachment is gone on the next reload. Record and photograph when there is a link, or expect to repeat it.

Each of the four forms opens with the last entry of its own kind, in a panel headed `Last Message | A. Wicki | 21:04:37Z` — what it was, who filed it and when. ATC spells the call out as `RCVD ZURICH INFO FREQ 124.700 MHz SQ 2450` with the wording beneath; Ballast shows the drop or the inventory; Resources the crew and power state; Other the note. Where nothing of that kind has been reported yet, the panel simply reads `-`.

The reporter is named because the entry may well have come from the other pilot's device: the panel is read from the table, not from what this device happens to remember. A follow-up call never has to be reconstructed from memory, and never depends on who made the last one.

8. WhatsApp

ATC, Resources and Other each list the recipients individually at the foot of the form, under *Also send to WhatsApp* — the entry always goes to the table, and WhatsApp is the addition, one chip per person, tapped on and off. Ballast entries never go to WhatsApp.

On ATC the coordinator sits on a line of its own above the rest of the list, so the one recipient that matters is never lost among the others.

Each recipient has a pill of its own with a check circle in front of it, filled when it is on and clearly outlined when it is off — an earlier version drew the off state so faintly that a list of three looked like a list of one. The label counts them, *Also send to WhatsApp · 3*, so an empty list is visible as an empty list rather than as a bug. Pills are matched by number rather than by position — an earlier version matched by index and silently dropped the coordinator, which is the sort of thing that is only noticed when a message does not arrive.

The **ATC Coordinator** is listed first and only on ATC, and is the only one ticked there by default — an ATC call belongs with the coordinator, not with the whole crew. On Resources and Other the coordinator does not appear and everybody else is ticked by default. Each choice is remembered per message type, so the pattern you settle into is the one you keep.

Recipients are defined in the setup, up to eight, each with a name and a number in international form with digits only, for example `41791234567`.

Above them sits a ninth, separate entry: the **ATC Coordinator**. It is offered on ATC entries only, as its own checkbox above the general one, and it is ticked by default there — an ATC call normally goes to the coordinator whether or not the rest of the list is in play. The two checkboxes are independent, so an ATC entry can go to the coordinator alone, to the list alone, to both, or to nobody. Resources and Other never see the coordinator.

With the box ticked, posting first writes the entry as usual, then opens a sheet with the finished message and one button per recipient. An ATC entry reads:

```
*ATC | HB-QWV*
Station: ZURICH INFO
FREQ: 124.700 · SQ: 7000
Msg: QNH 1013, Ops Normal
21:04Z · 484532N 0072345E · 2340 ft · TC 293 · 12.3 kn
https://maps.google.com/?q=48.75890,7.39580
```

The first line is bold in WhatsApp, and the last is the position, which WhatsApp turns into a tappable link with a map preview card of its own.

The small confirmation that follows a post now sits at the right edge above the button instead of on top of it, and the mark in the footer holds still: it says where the queue stands, not whether a poll happens to be running this second. Green double check for sent, straw single check with the count for waiting, red cross when an attempt was refused.

The post button carries the whole exchange. The moment it is pressed it greys out, locks and reads *… wait for confirmation*, so a second tap cannot post the same entry twice. The entry appears in *Last Message* as soon as it is written. Then the button becomes the answer for three seconds — pale green *Posted to CC Notion* when the row reached GitHub, pale yellow *Queued for later delivery (no internet connection currently)* when it is waiting, pale red *Failed* with the reason when something else went wrong — an expired token, a repository that is not there. The entry itself is never lost either way; the band only says where it stands.

Sending is immediate. With a box ticked, pressing *Post to CC Notion* writes the entry and opens WhatsApp with the message already in the chat — no preview, no list, no extra tap. The tick survives from one message to the next, so a run of ATC calls to the coordinator is one button each. Only the send button inside WhatsApp is still yours to press; no browser can press that one. With more than one recipient the first opens straight away and a short list of the rest appears behind it, since WhatsApp takes one chat at a time.

The layout is yours

The setup holds one template per message type under *WhatsApp message layout*. Placeholders in braces are substituted, and lines behave in three ways:

- `{key}` — a line whose placeholders are all empty is dropped, so a field nobody reported costs nothing.
- `{!key}` — the line is kept whatever happens, and an empty value prints as `-`. The Msg line uses this, so every message shows what was said even when nothing was.
- A line made of nothing but values and separators, like `{time} · {pos} · {alt} · {tc} · {speed}`, keeps its own separator and simply loses the parts that are missing. Without a fix it collapses to the time alone, with no stray dots left behind.

Reset to default puts the three templates back.

Placeholder	Value
`{type}` `{callsign}` `{flight}` `{reporter}`	who and what
`{dir}` `{station}` `{freq}` `{squawk}` `{msg}`	the ATC fields
`{pic}` `{resting}` `{battery}` `{fuelcell}` `{solar}`	the Resources fields
`{note}`	the free remark
`{time}`	UTC as `21:04Z`
`{pos}`	degrees, minutes and seconds, `484532N 0072345E`
`{posdm}`	degrees and minutes only, `4745N 00732E`
`{alt}`	GPS altitude in feet, rounded to ten

Placeholder	Value
`{tc} {speed}`	track as `TC 235` and ground speed as `12.4 kn`
`{maps}`	link to the position on Google Maps

Without a position fix `{pos}`, `{alt}` and `{maps}` are empty: the map line disappears and the time line keeps just the time.

The row records `whatsapp` = `yes` and `whatsapp_to` with the names, so the table shows which entries were meant to go out. Whether they were actually sent happens inside WhatsApp and cannot be recorded here.

8a. Master and followers

The master is whoever knows the master password. There is no other rule and no default.

Two passwords, doing two different jobs:

Password	What it does	Who has it
1234	opens the settings screen	everybody in the crew
5678	makes a device the master	one person, normally the PIC

The first time the app starts on a fresh device it asks the question outright: *Is this the master device?* Entering **5678** makes it the master. Choosing *Join as follower* — or getting the password wrong three times — makes it a follower. A device that was set up through an invitation link is a follower without being asked, because the link came from a master.

Nothing is assumed. A device that has not answered the question is not yet either, and the question comes back on the next start until it is answered.

Later changes go the same way: in the settings the role can be switched to Follower freely, but switching to Master asks for **5678** again. And because two masters would overwrite each other's setup, each published setup carries the identity of the device that wrote it; a master that sees a different one in the file says so on screen.

One device owns the flight setup and is the **Master** — normally the PIC's iPad. The others are **Followers**.

Menu → *Share setup with another device* produces a link carrying the flight, the pilots, the ballast figures, the Reporting Webhook, the WhatsApp recipients and the message templates. Opening it on the other device shows what is about to be applied and asks for a yes; on yes that device takes the whole setup and marks itself a Follower. A checkbox decides whether the GitHub token travels with the link — with it the other device can send at once, without it the token has to be typed there. Send a link containing a token the way you would send a password.

What the link deliberately leaves alone: the reporter name, the personal-or-shared device mode, the colour scheme and the confirmation tone. Those belong to the device.

Every time the master saves settings it publishes them to `data/_setup.json`, and followers apply the change within about thirty seconds with a note on screen. Rename a pilot or start a new flight on the master and the crew follows without anyone opening a setup screen. Exactly one device may be the master; two would overwrite each other's publication.

8b. What carries over

Every form arrives holding what was reported last time, greyed and dashed: the station, frequency and squawk of the last call, the kilograms of the last drop, the bags and litres of the last inventory, the last battery, methanol and oxygen readings. Selections — sand or water, PIC and resting, fuel cell, solar cell, crew on oxygen, the ready-ballast percentage — arrive at the same setting they were left in.

The defaults come from the table, not from the device. They are read from the last entry of that kind on the whole flight, whoever made it and on whichever iPad. Hand the reporting over mid-flight and the second device opens with exactly what the first one last reported, within the five seconds it takes the table to arrive. Only free text is never carried over — remarks, notes and the ATC message are typed each time.

Touching any greyed field in a form accepts all of them at once, and nothing is asked again at posting. Notes and messages are never carried over: repeating yesterday's wording verbatim is worse than typing it again.

The footer names the last entry, its reporter and its kind, plus the state of the upload — `last msg 21:04:37Z/AW/Ballast | upload ✓` when everything has reached GitHub, `✗ 3` when three are still queued.

9. Sending, and what happens without a link

There is nothing to switch on. Pressing **Post to CC Notion** writes the entry to the device and sends it straight away to GitHub and, independently, to the Reporting Webhook. The webhook receives one top-level JSON object per press, using the same entry schema as a row in the GitHub JSON file. The `type` field identifies the path: `ATC`, `Ballast`, `Resources` or `Other`. Automatic `POS` heartbeat rows are not sent to the webhook because they are not created by a press on this button.

If there is no connection the GitHub copy is held in the main queue — the header shows how many are waiting — and that queue goes out on its own as soon as the link returns, either on the browser's online event or on the next five-second cycle, whichever comes first.

Webhook delivery has its own local retry queue. Each queued item keeps the URL that was active when it was posted and retries with exponential pauses from five seconds up to five minutes. If GitHub succeeds while the webhook is still waiting, the post button turns yellow and says so. The request body is JSON but deliberately carries no custom `Content-Type` header: Zapier advises browser clients to omit it so the Catch Hook does not reject the CORS preflight.

Use `id` as the unique key in the Zapier/Notion action and upsert instead of append. A retry can otherwise create a duplicate in the rare case where Zapier accepted a request but the device lost the response before it could remove the local queue item.

Order is never in doubt: the file is always written as the complete set of rows sorted by `ts_utc`, so a queued entry lands in its correct place in the sequence rather than at the end. An entry made at 21:04 and sent at 21:11 still sits between 21:03 and 21:05 in the table.

Print the log builds an A4 portrait printout in its own window: you pick which of the four kinds it should contain, so an ATC-only log for the authority is one tap away. Entries run chronologically and numbered, each with its time, kind, content, reporter, position and track, one under the other as a table.

Column widths come from a colgroup rather than from the first row, so a page that opens with a full-width date heading lays itself out exactly like every other page.

A gas balloon flight can run for days, so the printout is broken by date. Each day opens with its own heading, in bold against the left edge of the table — `Wednesday, 20 May 2026` — and the numbering runs straight on across the break, because the entries are one flight and not three. The heading beside the callsign carries the span: `Wed 20 May 2026` for a single day, `Wed 20 May 2026 - Fri 22 May 2026` when it went further. Dates follow the UTC clock, like every time in the log, so a launch at 23:40 local does not appear to belong to the wrong day. A day heading never stands alone at the foot of a page — it travels with the first entry under it. Every page carries the flight in its heading with the mark on the right, and a footer reading `printed 2026-08-13 09:45Z by A. Wicki · v260813-03` on the left and `Page 2 / 3` on the right. After the last entry the table closes with `[no further log entries]`, so a printout can be seen to be complete.

Pagination is measured: the app takes the real height of a sheet, subtracts the heading, the footer and the margins, and fills each page with as many rows as actually fit. Where a browser gives no measurements it falls back to a fixed count rather than putting one entry per page.

Send now in the menu forces an attempt, and *Reload from GitHub* pulls the table without writing. Neither is needed in normal use. *Export the table* asks for the format first — CSV or JSON — and then for the destination: *Download* puts the file in the browser's downloads, *Share…* hands it to the system share sheet, which on an iPad is the way into Files, Mail or a chat. Share only appears where the browser supports handing over files.

10. Several devices at once

Every row carries `reporter` (who entered it), `device` (which iPad) and `source` (`app` for this tool).

Sending is a merge, never an overwrite: the app reads the file on GitHub, merges it with what is held locally by `id`, recalculates the ballast column over the whole set, and writes the result back. If another device wrote in the meantime, GitHub rejects the write and the app retries with the fresh version, up to three times.

The table is played back every five seconds. A drop made on one iPad shows up on the other within a few seconds, and the ballast figure accounts for both. A device that joins mid-flight, or whose local copy was cleared, gets the whole flight back with *Reload* in the log screen.

Polling that often is affordable because the app sends a conditional request: when nothing has changed, GitHub answers 304 with no body, and a 304 does not count against the hourly limit of 5000 requests. Only an actual change costs a request. Polling pauses while the app is in the background.

Tally rows. Anything appended to the same JSON file appears in the app and in the ballast calculation, provided it carries at least `id`, `ts_utc` and `type`. Give Tally-sourced rows `reporter: "Tally"` and `source: "tally"` so it is visible where they came from. A ballast row may be relative (`ballast_delta_kg`, negative for a drop) or an inventory (`ballast_action: "count"` with `total_ballast_kg`, which resets the running total).

Deletions are shared through `data/<flight-id>.deleted.json`, holding the ids that were removed with the time and the person who removed them. Every device applies that list, so a deleted row does not reappear from another iPad's copy, and the main table stays clean.

11. The log

Log in the menu opens it: the whole flight, newest first, with time, content, reporter and a dot showing whether the row has been sent. Nothing else — transfer and export moved into the menu.

Each row carries its transmission state at the right edge: a green double check when it has reached GitHub, a straw single check while it is waiting for a connection, and a red cross when an attempt was refused — an expired token, a repository that is not there. A cross is worth acting on; a single check only means the link is not up yet.

The ticks follow the queue on their own. When a batch finally goes out, every straw check in the list turns green within the second, without leaving the log or pulling to refresh. The list is only redrawn when the count of waiting or failed rows actually changes, so it does not fight your scrolling.

Tapping an entry opens it for editing. The editable fields depend on the type — kilograms and remark for a drop, bags, water and ready ballast for an inventory, station through message for ATC. Saving stamps `edited_at` and `edited_by`. The ballast column is recalculated over the whole flight after every edit and every deletion, so the figures stay consistent wherever in the sequence the change was made.

Deleting asks twice, states what is being removed and who recorded it, and warns that the deletion is shared with the other devices.

12. What the app writes

```
data/<flight-id>.json          the table
data/<flight-id>.csv           the same rows as CSV
data/<flight-id>.deleted.json  ids that were removed
```

Field	Content
`flight_id`, `callsign`	from the setup
`seq`	running number on the capturing device
`device`	device tag
`reporter`	who made the entry
`source`	`app`, or whatever an external writer sets, e.g. `tally`
`ts_utc`, `ts_local`	the same instant in UTC and with local offset
`type`	`Ballast`, `ATC`, `Resources`, `Other` — the values used on the Tally form
`pos_lat`, `pos_lon`, `alt_ft`	position of the reporting device in short notation, `484532N` / `0072345E`, and altitude in feet
`tc_deg`, `speed_kt`	track over ground in degrees and ground speed in knots, one decimal
`lat`, `lon`, `alt_gps_m`	the same fix in decimal degrees and metres, for map links
`gps_acc_m`, `gps_age_s`, `gps_fix`	quality of the position at the moment of the entry
`device_mode`	`personal` or `shared`
`ballast_action`	`drop` or `count`
`ballast_medium`	`sand` or `water` on a drop
`ballast_delta_kg`	kilograms thrown, negative
`ballast_abs_kg`	on board after this entry, recalculated across all devices
`sand_kg`, `water_kg`, `total_ballast_kg`	inventory result

Field	Content
`sand_left_kg`, `water_left_kg`	what is left of each after this entry
`tally_diff_kg`	counted total minus what was expected at that moment
`inv_bags`, `inv_ready_pct`	what the inventory was built from
`atc_dir`	`RX` received, `TX` sent
`atc_station`, `atc_freq`, `atc_squawk`, `atc_msg`	message content
`crew_pic`, `crew_rest`	crew state at the moment of the entry
`res_battery_pct`, `res_fuel_cell`, `res_solar`	battery, fuel cell and solar cell as reported on a Resources entry
`res_methanol_pct`	methanol level in per cent
`res_o2_crew`	whether the crew is on oxygen
`res_o2`	reserve as `560 l (70%)`
`res_o2_liters`, `res_o2_pct`	the same two figures on their own
`res_o2_detail`, `res_o2_bars`, `res_o2_inuse`	per cylinder, and which one is in use
`note`	free remark
`whatsapp`, `whatsapp_to`	set when the entry was marked for WhatsApp, and to whom
`attachments`, `attachment_paths`	how many pictures an Other entry carries, and where they were written
`edited_at`, `edited_by`	set when a row was changed after the fact
`id`	UUID, the stable key for Notion

`type` uses the canonical values `Ballast`, `ATC`, `Resources`, `Other`, and `POS`. Legacy `Ressources` rows are normalized to `Resources` when the app loads them, so existing flight data remains usable and is rewritten with the corrected value on the next synchronization.

13. Polling from Notion

GitHub API — immediate, recommended

```
GET https://api.github.com/repos/<owner>/<repo>/contents/data/<flight-id>.json
Authorization: Bearer <token>
Accept: application/vnd.github.raw
```

Poll `<flight-id>.deleted.json` the same way and archive the matching rows in Notion. Use `id` as the unique key and upsert; the file always holds the complete set of rows, so appending produces duplicates.

raw.githubusercontent.com — public repositories only, delayed

```
GET https://raw.githubusercontent.com/<owner>/<repo>/main/data/<flight-id>.csv
```

Behind a CDN with roughly a five minute cache. Fine after the flight, too slow during it.

14. Limits worth knowing before takeoff

- **GPS in the background:** once the app is not in the foreground and the display locks, iPadOS suspends location updates. The app holds a wake lock while it is active. This records events, not a track — use a dedicated logger for a gapless trace.
- **Altitude:** `alt_gps_m` is GPS altitude above the WGS84 ellipsoid, not the altimeter reading. The altimeter governs what is reported to ATC.
- **Rate limit:** with two iPads and a Notion poll on the same token, the hourly budget is comfortable as long as the conditional requests keep returning 304. Frequent writes are what cost — each post is roughly four requests.

- **The passwords are 1234 and 5678 and both live in the source.** They guard against a mistaken tap in the basket and against a second device declaring itself master, not against anyone who reads the file. Change `PASS` and `MASTER_PASS` in `index.html` for different ones — and if you do, tell the crew, because a device that cannot answer the master question ends up a follower.
- **Token on the device:** stored unencrypted in the browser. If the iPad is lost, revoking the token is enough. To avoid it entirely, put a Cloudflare Worker in front holding the token server side and point the API host in the code at the Worker.
- **Webhook URL on the device:** the Zapier Catch Hook URL is stored unencrypted, is published by the Master in `data/_setup.json`, and can be carried in the setup invitation link. Treat both the URL and an invitation containing it like a secret; anyone who has it can submit to that Zap.
- **The pill floats inside the app only** on phones and tablets — see section 4. Slide Over on iPad and Pop-up view on Samsung are the routes that put the whole window on top; document picture-in-picture on a laptop is the only true always-on-top.

13c. Place names

Off by default, and a setting of its own under *Resources*. Switched on, each entry's position is turned once into something a reader can place — `near Szeged/HU` — which then appears under the position in the WhatsApp message, in the log, in the printout and in a `place` column.

Two things to weigh before switching it on. The lookup goes to OpenStreetMap's public service, so the coordinates of the entry leave the device; and it only works with a link. There is no offline place database that would fit in a web app. Failures are silent by design: the entry is written immediately and the place is filled in afterwards if an answer arrives, so a lost lookup costs nothing but an empty field.

Answers are cached on a coarse grid — about a kilometre — so a long flight costs a few dozen requests rather than one per entry, which keeps well inside what the service asks of its users.

14a. Language

Device mode, reporter name, night colour, confirmation tone and language sit above the lock, because they belong to this device and every crew member may want different choices. Followers can change those personal settings, but only the Master can unlock the shared flight setup.

Every label, button, hint, dialog and message in the app follows the switch. **Only the interface changes:** everything written to GitHub, sent to the Reporting Webhook or sent to WhatsApp stays English: the column names, the values in `type`, the message templates. A table that changed language depending on who happened to make the entry would be unusable, and the ATC coordinator should not have to guess which language the next message arrives in.

14b. Beginning a new flight

Begin new flight appears **only on the master**, at the foot of the unlocked settings, and asks for the master password `5678`, then for the name of the new flight, then twice for confirmation — naming how many entries the old flight holds and how many of them are still queued.

Nothing is moved and nothing is overwritten. The old flight's files are named after it, so they simply stay where they are; they are recorded in `data/_flights.json` with the time they were closed, the number of rows and who closed them, which gives you an index of the season. Before closing, whatever is still queued is sent one last time. If there is no connection and rows are still waiting, the app says so and asks again before going on — those rows would be lost, because clearing the local table is what makes the new flight start empty.

The new flight ID is published to the followers along with the rest of the setup. A follower that sees the flight change files its own table away under `bsk.archive.<flight>` on the device and then clears it, so nobody carries yesterday's entries into today's flight under the new name.

The first entry of a flight belongs to the master. A follower trying to make it is told *New flight must be initiated by the master device*; the master is reminded once that the flight has been opened and asked to look over the settings, and the setup screen opens for that. From the second entry on, anyone reports.

15. Version

The build stamp sits in the bottom right of every screen, in the form `v<YYMMDD>-<nn>` — the date written backwards, then a counter that restarts at 01 each day and goes up with every build released that day. The date leads, so `v260813-01` is newer than `v260812-26` despite the smaller counter. `v260813-09` is the ninth build of 13 August 2026.

It lives in two places that must be kept in step: the `APP_VERSION` constant near the top of the script in `index.html`, and the cache name `V` in `sw.js`. `readme.html`, `setup.html` and the PDFs are generated from `README.md` and `SETUP.md` and are regenerated on every build, so the printed manual is never behind the app. Bumping the cache name is what forces the service worker to fetch the new files rather than serve the old ones, so a build with an unchanged cache name may not reach a device that already has the app installed.

16. License

Custom license, © 2026 Wicki Aero GmbH.

Read the full licence — it opens in a window of its own, and the same text sits in `LICENSE` in the repository.

In short: use it and host it as it is, for yourself, your club or your school, free of charge. Do not modify it, translate it, republish a changed version of it, strip the attribution from it, or build a competing product on it, without asking first — balthasar@wicki.aero. Any public copy keeps the copyright notice and a visible link back to the source repository.

Two things the licence spells out that are easy to get wrong:

Configuration is not modification. Entering your flight, crew, ballast, Reporting Webhook, WhatsApp and GitHub settings, switching language or colour scheme, editing the message templates in the setup, and dropping your own logo file next to the app are all ordinary use. You are meant to do all of that.

The flight data is yours. The JSON, CSV and media files the app writes into your repository are your records, not part of the licensed software.

Third-party components: none. No libraries, no fonts, no frameworks, no trackers, nothing from a CDN. The app talks to the GitHub API with your token, to the configured Zapier Catch Hook, to OpenStreetMap when place names are enabled, to the browser's geolocation service, and to the wa.me links you tap. That is the whole list, which is also why the reporting form keeps working with no signal and why there is nothing to keep up to date but the app itself.

17. Files

```
index.html                  the complete app
manifest.webmanifest        Home Screen installation
sw.js                       offline cache
readme.html                 this document, opened from the menu
Basket-Reporting-Manual.pdf this document, for printing
SETUP.md / setup.html       the GitHub and Notion setup guide
Setup-data-repository.pdf   the setup guide, laid out for printing and handing over
LICENSE / license.html      the licence
favicon.ico                 multi-resolution favicon
icons/                      app icons and favicons
data/                       destination folder for flight files
```